

Rich I/O-centric Derived Quantity Operators for Advanced In Situ Processing

Hua Xu, Jaime Cernuda,
Anthony Kougkas, Xian-He Sun
Illinois Institute of Technology, Chicago, IL
hxu40@hawk.illinoistech.edu,
jcernudagarcia,akougkas,sun@illinoistech.edu

Ana Gainaru, Scott Klasky
Oak Ridge National Laboratory
Oak Ridge, TN
gainarua,klasky@ornl.gov

Michela Taufer, Nigel Tan
University of Tennessee
Knoxville, TN
mtaufer@utk.edu
ntan@lanl.gov

Kevin Assogba, M. Mustafa Rafique
Rochester Institute of Technology
Rochester, NY
kta7930@g.rit.edu,mrafique@cs.rit.edu

Bogdan Nicolae
Argonne National Laboratory
Chicago, IL
bnicolae@anl.gov

Josh Teves, Curtis Brinker,
Line Pouchard, Jay Lofstead
Sandia National Laboratories
Albuquerque, NM
jbteves,cjbrin,lcpouch, gflofst@sandia.gov

Abstract—Scientific simulations produce increasingly large datasets that surpass practical limits for frequent input and output operations. Although performing analysis directly as data is produced (in situ) can mitigate this problem, embedding such analysis routines into complex scientific codes is often impractical or inflexible. We address this by introducing derived quantity operators that provide a modular and pluggable framework within high-level I/O libraries, enabling sophisticated in situ analysis to be performed transparently during data output with minimal modifications to the host application. The operators bind to persistent hierarchical storage through operator-declared metadata, allowing cross-timestep and cross-run analysis to be carried out seamlessly within the I/O path.

We demonstrate the capabilities of our operators by integrating correctness validation and important scientific metrics, traditionally executed after simulation runs, directly into the I/O pathway as derived quantities. Our implementation illustrates how complex, non-mathematical analyses such as runtime validation can be efficiently performed during simulation, enabling early detection of errors. Our work broadens the range of in situ analyses achievable through derived quantity operators, promoting more flexible, intelligent, and reliable scientific workflows.

I. INTRODUCTION

As simulation fidelity and data volume grow, the limitations of traditional I/O-based workflows have become more pronounced. Data sizes in high-fidelity simulations have expanded to the point where writing output as often as desired is no longer feasible. This has driven interest in in situ analysis [1], which processes data in place during simulation and writes only the results. This work builds on that concept by introducing a pluggable operator framework that leverages the ability of high-level I/O libraries to intercept data and perform analysis in place of, or alongside, writing to storage.

Central to in situ analysis are *derived quantities*: values computed from simulation output that drive scientific insight, such as vortex-core indicators, divergence metrics, and temporal derivatives. We classify derived-quantity operators into two categories based on their data requirements. *Independent operators* compute a derived quantity solely from the current

timestep ($y = f(x)$); these cover the lightweight reductions that existing I/O-centric frameworks support well. *Dependent operators* compute $y = f(x, g(x'))$, where $g(x')$ is a transformation of data from a previous timestep or a prior run. Many scientifically critical analyses fall into this second class: binary integrity checking against a validated reference run [2], [3], [4], cross-run numerical comparison of how precision or compiler choice shifts a physical field [5], and temporal diagnostics that need a sliding window of snapshots within a run [6], [7], [8].

Despite the clear need, no existing framework supports dependent operators within the I/O path. Current I/O-centric derived-quantity frameworks [9] provide no mechanism to access results from previous timesteps or runs. Streaming transports such as ADIOS2 SST [10] expose in-flight data across steps within a running simulation but do not retain cross-run persistent state accessible to operators after run completion. Without persistent, operator-accessible state, every dependent analysis falls back to the post-hoc model: both runs write their output to the parallel file system, a separate program rereads both, and error norms are computed offline; this I/O cost alone can consume more than half the total time to scientific insight at scale [11].

The difficulty in supporting dependent operators stems from three compounding challenges. First, historical state cannot reside on the parallel file system: at the access latency of a PFS, retrieving reference data inline during simulation output negates any benefit over post-hoc processing. Second, operators differ fundamentally in their data requirements, so a rigid built-in reduction set is insufficient; a generalizable, extensible operator model is required. Third, placement and prefetch decisions must be driven by operator-declared metadata, a binding no general-purpose hierarchical storage manager provides.

We address these challenges with a framework that embeds *dependent operators* directly into the I/O path and manages cross-timestep and cross-run state through an operator-aware

metadata layer backed by the I/O manager. Operator-declared metadata drives placement and prefetch decisions, keeping reference data at low latency. We demonstrate three concrete operators spanning both classes: a Q-criterion operator (independent) that detects vortex cores from the current-timestep velocity field; a Merkle-tree [12]-based checksum operator (dependent) for binary integrity verification across runs; and a stencil operator [13] (dependent) for in-run temporal derivative diagnostics.

The **contributions** of this paper are as follows.

- 1) We define the independent/dependent operator taxonomy and introduce the first framework that supports dependent operators within the I/O path.
- 2) We investigate fully offline, hybrid, and fully online deployments through experiments, revealing each mode’s benefits and limitations.
- 3) We evaluate trade-offs across diverse hardware and software platforms, demonstrating how operational complexity interacts with system configurations.

II. BACKGROUND AND MOTIVATION

We describe how existing I/O-centric frameworks handle derived quantities, detail the dependent-operator scenarios they cannot support, and explain how the storage hierarchy enables the required persistent state.

A. I/O-Centric Derived Quantities

Recent work has demonstrated that I/O libraries can compute derived quantities during data output. Gainaru et al. [9] formalized three strategies within ADIOS2 [10]: *Store* (precompute and persist derived values alongside raw data), *Expression* (define derived quantities as expressions evaluated on read), and *Stats* (compute summary statistics such as min/max/mean during output). Their performance model quantifies the trade-offs between storage overhead and computation savings for each strategy. All three strategies, however, operate exclusively on current-timestep data, providing no mechanism for the dependent operators described below.

B. The Need for Dependent Operators

We detail the three scenarios that require historical context beyond what existing frameworks provide.

Binary integrity under divergence-prone dynamics: Detecting floating-point divergence at runtime requires checkpoint validation: comparing current simulation output, timestep by timestep, against a reference run of the same configuration that has been validated beforehand. Figure 1 illustrates the effects of ill-conditioned parameter regimes on the Gray-Scott model [14], where chaotic dynamics cause small perturbations to produce dramatically different outputs, motivating the need for runtime verification of simulation results against a previously validated dataset.

Continuous physics divergence across runs: Cross-run comparison operators produce field-level error norms between a reference run and a validation run, quantifying how alternate compilers or different hardware affect the scientific result [5].

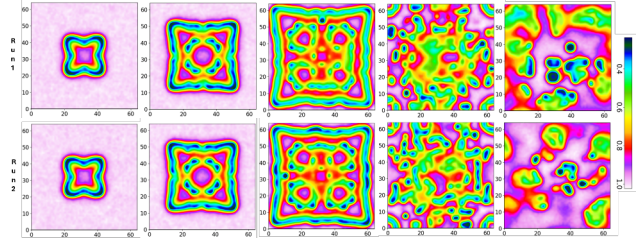


Figure 1: Reproducibility of the Gray-Scott reaction-diffusion model under ill-conditioned ($F = 0.014$, $k = 0.047$) parameter configurations. The ill-conditioned (chaotic) regime results in dramatically divergent pattern evolution despite identical initial parameters.

Temporal diagnostics without offline reprocessing: Temporal dependent operators maintain a sliding window of historical snapshots: feature tracking algorithms exploit overlap between consecutive snapshots to follow vortices, bubbles, or shocks through time [6], [7]; streaming uncertainty quantification updates autocorrelation functions and time-averaging error bounds from successive samples without retaining the full history [8]; and finite-difference temporal derivatives provide pressure-wave diagnostics in turbulence simulations.

C. Hierarchical Storage

Modern HPC systems incorporate multiple storage tiers (node-local NVMe and SSD burst buffers, remote burst buffers, and parallel file systems [15]) to bridge the widening gap between computational throughput and I/O bandwidth. Hierarchical storage management systems such as Ziggurat [16] target NVM-aware placement, while burst-buffer schedulers [17] focus on the staging-to-PFS path. Our framework builds on the I/O manager as a substrate but adds an operator-aware metadata layer that lets dependent operators drive placement and prefetch decisions; none of these systems exposes this binding.

III. DESIGN

Our framework extends the I/O library layer to support derived quantity operators, including those that access historical state. The design is guided by four objectives:

- 1) **Support dependent operators transparently:** Enable operators that access historical state from storage with minimal application code changes.
- 2) **Minimize application overhead:** Operator execution must not impact simulation performance on the critical I/O path.
- 3) **Leverage hierarchical storage:** Use tiered storage to keep historical data accessible at low latency for dependent operators.
- 4) **Ensure portability:** Operators must work across different platforms, file systems, and scientific applications through a standard I/O library interface.

Murmur3. Leaf digests are combined pairwise bottom-up, and the serialized tree becomes the derived variable's payload. Validation retrieves the reference tree through the metadata layer and traverses both trees breadth-first: matching subtrees are pruned, and element-wise comparison runs only on the leaves that survive, so cost scales with divergence rather than with data size.

C. Temporal Stencil Operator

A time derivative needs history, which a single-step derived variable cannot provide. The operator therefore attaches to the I/O manager as an ordinary client and reads the step-tagged objects the engine has already written, resolving the tag `step_N_rank_r` through the metadata layer. It keeps a per-rank window of five snapshots and applies fourth-order centered stencils for the first and second time derivatives [20], reporting the per-rank minimum, maximum, and L_2 norm.

D. Q-Criterion Operator

The Q-criterion marks regions where rotation dominates strain, the standard indicator of vortex cores. It depends only on the current step and returns a field of the input shape, so it fits the derived-variable model directly. Registered over the three velocity components, it forms the nine components of the velocity-gradient tensor and contracts it with its transpose,

$$Q = -\frac{1}{2} \frac{\partial u_i}{\partial x_j} \frac{\partial u_j}{\partial x_i} = \frac{1}{2} (\|\Omega\|^2 - \|S\|^2).$$

The derivatives use the host solver's compact finite-difference scheme [21], in which each directional derivative is a tridiagonal solve along a line of the domain. The reference implementation computes the same quantity post hoc, reloading the velocity components for every timestep from the file system.

V. EXPERIMENTS

A. Methodology

Hardware Platforms: We use three platforms, one of which is tested under three different configurations.

Platform 1 is designed to support hierarchical storage research. The cluster comprises a computer rack with 32 interconnected nodes. Each compute node is equipped with dual Intel Xeon Scalable Silver 4114 processors, 48 GB of RAM, and an NVMe PCIe x8 drive, alongside an SSD and an HDD. Platform 1 can be configured to use HDDs, SSDs, or NVMe for experiments.

Platform 2 consists of 21 nodes linked by interconnects offering 100 GB/s. Each compute node is equipped with an Intel Broadwell E5-2695v4 processor and 512 GB of RAM.

Platform 3 is the Frontier supercomputer, located at the Oak Ridge Leadership Computing Facility. The system provides a high-performance flash storage tier with 75 TB/s read and 35 TB/s write speeds and interconnects offering 100 GB/s.

Software: Our framework is publicly available on GitHub ¹. For our evaluations, we employ ADIOS2 [10] as the I/O

interface, Clio-core ² as the I/O manager, and SQLite v3.40.1 as the metadata storage medium. For our checkpoint validation operators, we use StateDiff [12] to generate the checksum for each timestep. All evaluations are repeated three times, and we present the mean and standard deviation of the results.

Gray-Scott Model: The Gray-Scott model is a reaction-diffusion system simulating chemical reactions and pattern formation in three dimensions. We leave most parameters at their defaults and vary two: the number of iterations (**steps**) and the grid size (L), where linear increases in L produce cubic increases in the number of cells.

Weather Research and Forecasting Model (WRF): WRF is an atmospheric modeling system used for both research and operational numerical weather prediction.

Xcompact3D: Xcompact3D [21] is a high-order CFD framework that uses the Immersed Boundary Method to represent complex geometries while preserving high-order discretization accuracy.

B. Case studies

We evaluate the three operators under three deployment modes of our framework, illustrated in Figure 5.

a) **Case 1 – Passive, Disk-Based I/O Layer:** Case 1 writes an output file for each application run. A separate distributed task later reads this file, and related functions compute derived quantities and augment the output with them. The application is then executed again, under the same configuration, to produce a second file. Finally, a program validates the second run by comparing its derived quantities against those from the previous run.

b) **Case 2 – Activating the I/O layer:** In this case, we enable the I/O library to use our operators. The reference run stores both the original variables and derived data in one file. A subsequent execution produces a second file containing its derived data for cross-run validation.

c) **Case 3 – Persisting data ownership:** Each written object stays addressable through the metadata database, so processing happens online inside MDLayer: historical data is compared against the current-step data still resident in memory. Because divergence is detected at the step it occurs, the run can be terminated early rather than completing and being discarded.

C. Synthetic Evaluation: Internal timings

Using the checkpoint checksum operator with Gray-Scott, we break execution time into its components (Figure 3). We separate five components: checksum generation; `Put()`, which associates an application data pointer with a variable definition but performs no I/O; application I/O, captured at `EndStep()`; checksum post-processing, the cost of computing checksums and augmenting the output in Case 1; and validation, the time to read checksums for both runs.

Case 3 with MDLayer utilizes tiered data management, resulting in significantly lower I/O time (1482 ± 15 ms)

¹<https://github.com/grc-iit/coeus-adapter>

²<https://github.com/iowarp/clio-core>

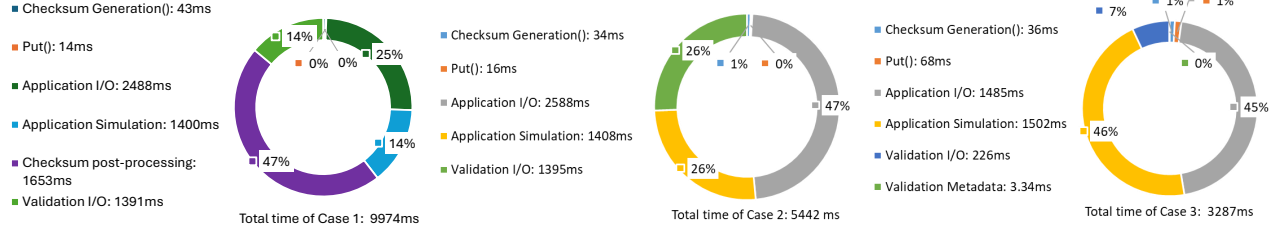
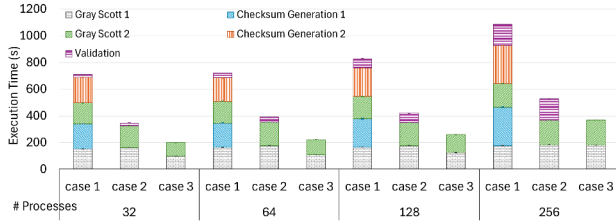
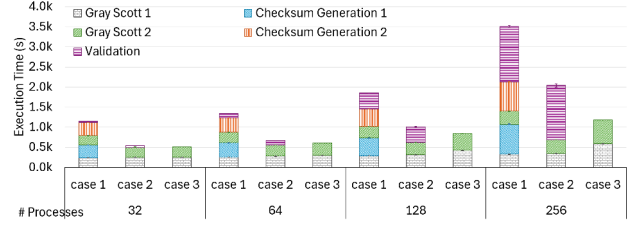


Figure 3: Component-wise execution time for Cases 1 to 3 at 128 processes on Platform 1, Gray-Scott. Case 3 achieves the lowest I/O time because the I/O manager absorbs writes into fast tiers; metadata query overhead is $\approx 0.1\%$ of per-step time.



(a) I/O scaling of platform 1



(b) Compute scaling of platform 1

Figure 4: I/O scaling (a) and compute scaling (b) on Platform 1 (SSD-backed PFS), 32–256 processes. Cases 1 and 2 scale poorly as validation re-read cost grows with data volume; Case 3 keeps validation overhead nearly flat by retaining data ownership in the hierarchy and retrieving the data via the metadata database for the validation.

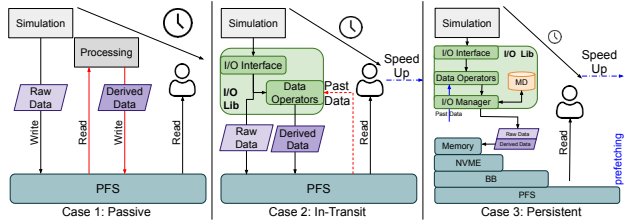


Figure 5: Deployment modes for the dependent operator. **Case 1 (Passive):** derived data is computed offline after the simulation, requiring full re-reads from PFS (red arrows), incurring two extra I/O passes. **Case 2 (In-Transit):** operators run inside the I/O library during the write path; derived data is stored alongside raw data. **Case 3 (Persistent):** the I/O manager retains data ownership across the full tier hierarchy with a metadata database and prefetching, enabling online processing.

compared to Case 1 (2494 ± 76 ms) and Case 2 (2588 ± 5 ms).

D. Comparison with In Situ Frameworks

We compare the cross-run validator against two systems spanning the dominant in situ design points: SENSEI [19], a backend-agnostic framework coupling analysis through an application adaptor, and ADIOS2 SST [10], a streaming transport routing in-flight data to a separate consumer. We run the Gray-Scott validator on Platform 1 (128 processes, $L = 512$, 30 steps), realizing comparison as a custom SENSEI adaptor and an SST consumer endpoint.

Table II: Cross-run checkpoint validation against external in situ baselines. Times in seconds; “Total” is simulation + validation.

System	Simulation (s)	Validation (s)	Total (s)
ADIOS2 SST	194.58 ± 2.41	27.41 ± 0.39	221.99
SENSEI	124.65 ± 1.31	26.37 ± 0.61	151.02
Case 2	117.12 ± 0.87	26.54 ± 0.25	143.66
MDLayer (Case 3)	94.48 ± 2.89	1.24 ± 0.15	95.72

Table II shows Case 3 dominates both baselines: fast-tier writes and in-place checksum reads cut validation time and simulation time.

E. I/O and Compute Scaling of System

To evaluate scalability, we run Gray-Scott with our checksum operator as a reference and validation pair. We conduct both I/O and compute scaling experiments across a range of 32 to 256 processes. For the I/O scaling, we ensure that each process handles the same data volume by adjusting the problem size: when the number of processes doubles, L is scaled by a factor of $\sqrt[3]{2}$, starting from $L = 428$ and increasing up to $L = 848$. This maintains a constant workload per process. For the compute scaling, we fix the I/O size per step by setting $L = 512$ and scale the total workload by doubling both the number of processes and the number of steps. This approach stresses the operator’s ability to handle increased parallelism and computational load while keeping the per-step I/O consistent.

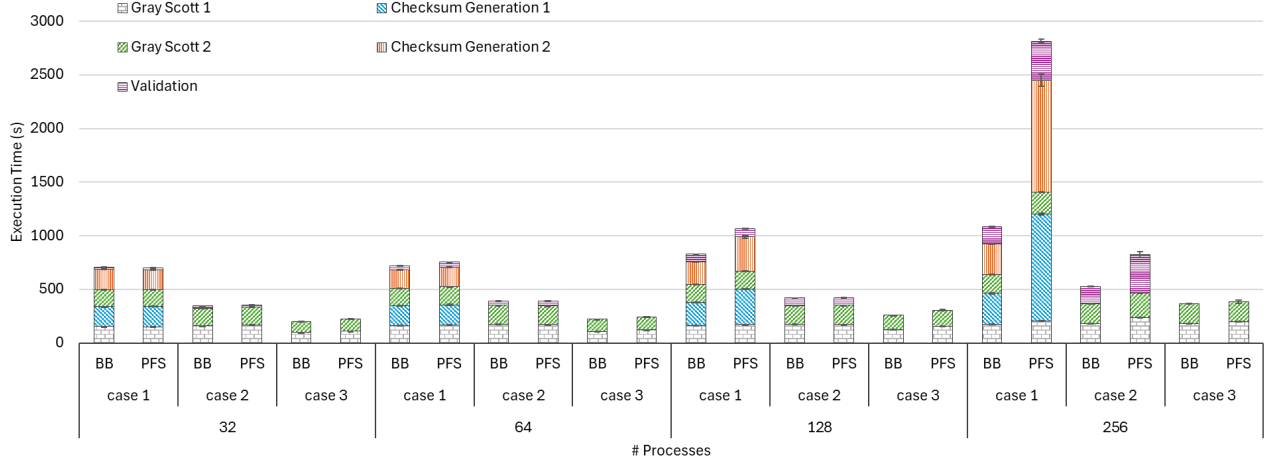


Figure 6: Execution Time on Platform 1 for I/O scaling: Performance scaling from 32 to 256 processes with a consistent I/O size per process, using a SSD-backed PFS as a remote burst buffer (BB) against an HDD-backed PFS. Case 3 keeps validation cost nearly flat regardless of the underlying storage hardware.

In the I/O scaling results (Figure 4a), Cases 1 and 2 exhibit limited scalability, primarily due to excessive I/O movement, an issue especially pronounced on a PFS backed by slower HDD storage. Data is transferred into userspace for checksum computation or verification, and then sent back to the storage system. This inefficiency is exacerbated when data divergence occurs. In the hybrid case, the impact on the application’s execution time is reduced because data transformations occur directly within the I/O path. While these transformations are fast, scalability is limited by validation overhead that grows with I/O volume at larger process counts. Finally, the online case proves to be the most efficient approach at larger scales. By retaining data ownership, asynchronously prefetching it closer to the user, and performing on-the-fly validations, Case 3 reduces and hides much of the I/O burden in traditional approaches.

In the compute scaling results (Figure 4b), when the number of processes reaches 128, the baseline and hybrid cases encounter bottlenecks in validation due to the significant overhead associated with reading small data chunks. Because the data remains under hierarchical management, the online case avoids frequent small-file read operations, reducing overhead in scenarios with a large number of processes.

F. Cross-architecture portability

We repeat the I/O scaling test on both HDD- and SSD-based storage to isolate the impact of storage bandwidth, with I/O size per process as in Figure 4. We emulate an application writing either directly to the PFS or through a remote burst buffer, represented by a remote SSD-backed OrangeFS setup that absorbs data quickly before asynchronously flushing it to the PFS.

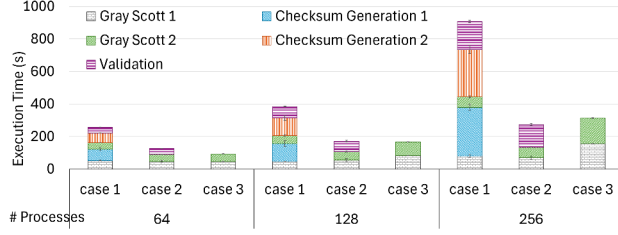
As shown in Figure 6, burst buffers (BBs) handle small and random I/O workloads (approximately 36.7 MB per process per step) far more efficiently than HDDs, resulting in improved overall performance. However, the file-level data layout can

lead to increased seek times on HDDs when the number of processes exceeds 128, amplifying the cost of operator execution and validation. For example, when the process count reaches 256, the validation time on the PFS increases substantially, reaching approximately 160 seconds when using burst buffers and 360 seconds with direct PFS access in both Case 1 and Case 2. In contrast, the validation overhead in Case 3 remains minimal (around 5 seconds) regardless of whether burst buffers or the PFS are used. This indicates that our hierarchical management, which combines in-memory buffering with batched I/O, effectively mitigates the impact of limited concurrent-access support on HDD-based systems.

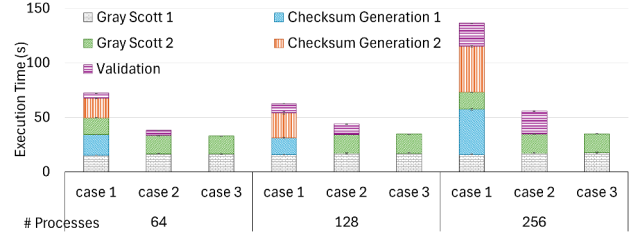
G. Cross-cluster portability

We repeat the I/O scaling evaluation on two additional platforms with distinct hardware and software configurations.

As shown in Figure 7, all three cases generalize across platforms, but the relative ranking shifts with hardware characteristics. On Platform 3, Case 3 consistently outperforms Cases 1 and 2: the 75 TB/s flash tier absorbs prefetched data at low latency, mirroring the Platform 1 SSD results. Platform 2 exposes a different regime. Without a local fast-tier storage system, the tier hierarchy collapses to RAM and a network filesystem; at 128–256 processes, evictions are driven to the network filesystem and metadata query contention accumulates, and as a consequence, Case 3’s total simulation time exceeds that of other cases. Notably, Case 3’s validation cost itself remains negligible at these scales; the excess comes entirely from metadata management and prefetch coordination, not from operator execution. This is not fundamental: raising the in-memory tier’s score or dedicating a metadata server would eliminate the network round-trips. These results confirm that the benefit of the persistent deployment mode requires either a local fast-tier storage system or platform-specific buffer tuning.



(a) Platform 2



(b) Platform 3

Figure 7: Cross-cluster portability of the checkpoint validation operator. For both the reference run (Run 1) and validation run (Run 2), Case 3 introduces minimal operator execution-time overhead, showing the operator framework generalizes across storage and interconnect technologies.

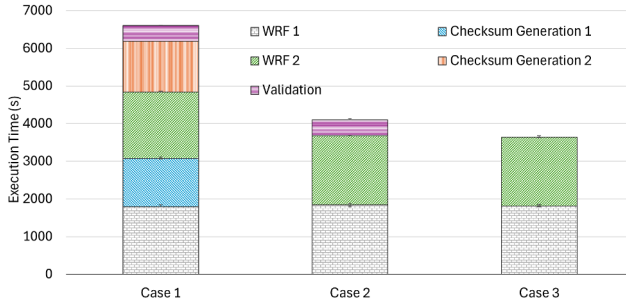


Figure 8: Execution time analysis of the Weather Research and Forecasting Model with the CONUS 2.5-km benchmark. Case 3 handles the multi-variable metadata query pattern and high-complexity I/O environment without measurable degradation.

H. Cross-application portability

WRF presents a substantially more demanding environment on Platform 1: it manages more than 190 variables and performs frequent, fine-grained I/O, a rigorous testbed for operator scalability under realistic atmospheric workloads.

We use the standard CONUS 2.5-km WRF benchmark [22], running two identical simulations (WRF1 and WRF2) on 180 processes to balance domain decomposition and minimize runtime noise. The benchmark covers a 1-hour simulation period with a 1-minute output interval, generating 60 output steps and producing a total of 166 GB of data.

As shown in Figure 8, Case 3 remains efficient as the number of variables and I/O operations grows, sustaining metadata queries across more than 190 variables per output step.

I. Cross-operator portability

1) *Q-criterion Operator*: The Q-criterion is a substantially more computation-intensive operator, demanding richer semantic context, tighter data dependencies, and sensitivity to data locality and metadata organization.

We study four Xcompact3D benchmarks spanning diverse flow regimes and I/O patterns: turbulent boundary layers, cylinder wakes, and cavity flows. These workloads span a broad range of computation-to-I/O ratios (4:1 to 7:1) and data

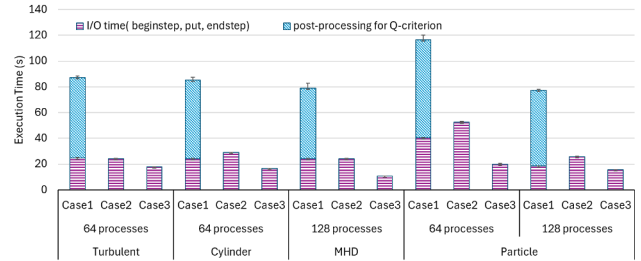


Figure 9: Q-criterion operator performance across four Xcompact3D flow regimes, showing the hierarchical management benefit extends to computation-heavy independent operators.

scales. For each benchmark, we measure I/O performance across different cases, as well as the post-processing time for the Q-criterion calculation in Case 1. Note that in Cases 2 and 3, Q-criterion computation is performed during the I/O phase and is therefore included in the I/O time.

As shown in Figure 9, Case 3 consistently achieves about 30% lower I/O time than Case 2 across all benchmarks. MD-Layer therefore handles computation-intensive, multi-variable derived quantities while still outperforming Cases 1 and 2 on I/O.

2) *Stencil Operator*: The 4th-order centered stencil we use spans a five-step window, so each emission must combine the current sample with four earlier ones. The conventional way to supply that history in Xcompact3D is an application-side five-slot ring buffer maintained inside the solver: a full pressure field per rank is retained in each slot, yielding a permanent per-rank memory footprint of five times the raw variable size, and every emission shifts the buffer, applies the stencil, and writes the result. We refer to this as the ring buffer approach. MD-Layer instead retains prior timesteps in the hierarchical tier and evaluates the stencil in a separate consumer attached to the I/O manager, which reads them back through the metadata database and leaves the solver itself untouched.

We exercise both modes across three Xcompact3D benchmarks, weak-scaled at 32, 64, and 128 processes. The window fills at the 5th timestep, after which the operator emits con-

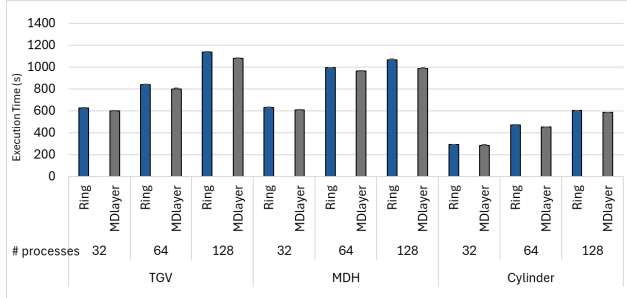


Figure 10: Stencil operator: in-solver ring-buffer baseline vs. MDLayer, on three Xcompact3D benchmarks at 32–128 processes. MDLayer reduces total execution time in all nine configurations; retrieving each timestep through the metadata index adds only $\approx 3\text{--}5\%$ to I/O time while moving the five-field-per-rank pressure window out of the solver.

tinuously until the end of the simulation; because the stencil is centered, each result is attributed to the middle step of its window and so trails the simulation by two output steps.

As shown in Figure 10, MDLayer reduces total execution time across all nine matrix cells. Our metadata system and hierarchical storage layer make the history essentially free to obtain: each timestep is retrieved once through the metadata database as it is produced, and the stencil, including that retrieval, adds only $\sim 3\text{--}5\%$ to the total I/O time, demonstrating that locating and reading raw fields through the metadata index incurs negligible overhead at production I/O sizes. Beyond the wall-time win, MDLayer moves the five-field-per-rank pressure window out of the solver, freeing a substantial fraction of per-rank memory that grows with grid size; the window is maintained by the consumer instead, off the application’s critical path. This extends our portability claim to operators with temporal data dependencies.

J. Discussion

Our evaluation covers both dependent and independent I/O-centric operators across three platforms, including a platform with two different hardware configurations, yielding four distinct experimental environments.

In small and frequent I/O workloads, metadata overhead becomes more noticeable. While the absolute overhead is low, its relative impact grows, revealing a regime where metadata enrichment offers diminishing returns. However, this can be mitigated through a number of existing solutions, such as metadata aggregation [23], [24].

VI. RELATED WORK

A lighter-weight alternative places analysis within the I/O library rather than coupling to a separate framework. Freeprocessing [25] demonstrates a complementary technique: function interposition on HDF5 and NetCDF calls transparently injects analysis tasks at the I/O boundary without any simulation source-code modifications. Hades [26] extends active-storage concepts to push computation closer to where data

resides. These designs confirm that embedding operators in the output path reduces overhead significantly, but none provides a mechanism to access data from prior timesteps or runs.

At a coarser granularity, workflow systems add monitoring and steering atop existing I/O. INSTANT [27] orchestrates in-situ workflows across distributed sites using ADIOS2 for data movement; MEOW [28] triggers processing steps from file-creation events. For data correctness, Rynge et al. [29] integrate end-to-end integrity checks into the Pegasus WMS, catching accidental or malicious corruption during inter-stage transfers. All of these approaches require externalizing data to storage or transferring it between workflow stages before any cross-step comparison can occur.

VII. CONCLUSIONS AND FUTURE WORK

In this study, we assessed the effectiveness of integrating complex in situ processes within the I/O layer. We evaluated three applications: Gray-Scott, WRF, and Xcompact3D, which exhibit distinct I/O patterns. The results demonstrate that the operator-based approach adapts effectively across all three applications, as well as across diverse hardware configurations and file systems, highlighting its versatility in handling heterogeneous I/O workloads.

For future work, we plan to introduce more advanced and versatile operators and support more complex in situ processing tasks, including in situ visualization, adaptive data reduction, and dynamic data placement strategies.

ACKNOWLEDGMENTS

This material is based upon work supported in part by the National Science Foundation (NSF), Office of Advanced Cyberinfrastructure (OAC), under Grant No. 2411318, and the U.S. Department of Energy (DOE), Office of Science, Office of Advanced Scientific Computing Research (ASCR), under Award Number DE-SC0023263. This research used in part resources of the Oak Ridge Leadership Computing Facility (OLCF), which is a DOE Office of Science User Facility. This research also used in part computational resources provided by Sandia National Laboratories, a multi-mission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC (NTESS), a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy’s National Nuclear Security Administration (DOE/NNSA) under contract DE-NA0003525. This written work is authored by an employee of NTESS. The employee, not NTESS, owns the right, title, and interest in and to the written work and is responsible for its contents. Any subjective views or opinions that might be expressed in the written work do not necessarily represent the views of the U.S. The publisher acknowledges that the U.S. Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this written work or allow others to do so, for U.S. Government purposes. The DOE will provide public access to results of federally sponsored research in accordance with the DOE Public Access Plan. Government.

REFERENCES

- [1] D. Thompson, N. D. Fabian, K. D. Moreland, and L. G. Ice, "Design issues for performing in situ analysis of simulation data," *Technical Report SAND2009-2014, Sandia National Laboratories, Tech. Rep.*, vol. 2009, no. 2014, pp. 1–10, 2009.
- [2] B. Geyer, T. Ludwig, and H. von Storch, "Limits of reproducibility and hydrodynamic noise in atmospheric regional modelling," *Communications Earth & Environment*, vol. 2, no. 1, p. 17, 2021.
- [3] O. Mesnard and L. A. Barba, "Reproducible and replicable computational fluid dynamics: It's harder than you think," *Computing in Science & Engineering*, vol. 19, no. 4, pp. 44–55, jul 2017.
- [4] S. Shanmugavelu, M. Taillefumier, C. Culver, O. Hernandez, M. Coletti, and A. Sedova, "Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications," in *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '24. IEEE Press, 2024, pp. 170–179.
- [5] F. Hoerold, I. R. Ivanov, A. Dhruv, W. S. Moses, A. Dubey, M. Wahib, and J. Domke, "RAPTOR: Practical numerical profiling of scientific applications," arXiv preprint arXiv:2507.04647, 2025, preprint. [Online]. Available: <https://arxiv.org/abs/2507.04647>
- [6] E. P. N. Duque, D. E. Hiepler, S. M. Legensky, and C. P. Stone, "In-situ feature tracking and visualization of a temporal mixing layer," in *SC Companion: High Performance Computing, Networking, Storage and Analysis*. IEEE, 2012, pp. 1593–1593.
- [7] S. Dutta, T. Turton, D. Rogers, J. M. Musser, J. Ahrens, and A. S. Almgren, "In situ feature analysis for large-scale multiphase flow simulations," *Journal of Computational Science*, vol. 63, p. 101773, 2022.
- [8] S. Rezaeiravesh, R. Vinuesa, and P. Schlatter, "In-situ estimation of time-averaging uncertainties in turbulent flow simulations," *Computer Methods in Applied Mechanics and Engineering*, 2024, arXiv:2310.08676.
- [9] A. Gainaru, N. Podhorszki, L. Dulac, Q. Gong, S. Klasky, G. Eisenhauer, A. Kougkas, X.-H. Sun, and J. Lofstead, "To derive or not to derive: I/O libraries take charge of derived quantities computation," in *2024 IEEE 36th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2024, pp. 105–115.
- [10] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Geraschewski, K. Huck, A. Huebl, M. Kim, J. Kress, T. Kurc, Q. Liu, J. Logan, K. Mehta, G. Ostrouchov, M. Parashar, F. Poeschel, D. Pugmire, E. Suchyta, K. Takahashi, N. Thompson, S. Tsutsumi, L. Wan, M. Wolf, K. Wu, and S. Klasky, "ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management," *SoftwareX*, vol. 12, p. 100561, 2020.
- [11] H. Ather, J. L. Bez, B. Norris, and S. Byna, "Illuminating the i/o optimization path of scientific applications," in *High Performance Computing*, A. Bhatele, J. Hammond, M. Baboulin, and C. Kruse, Eds. Cham: Springer Nature Switzerland, 2023, pp. 22–41.
- [12] N. Tan, K. Assogba, W. J. Ashworth, B. Bogale, F. Cappello, M. M. Rafique, M. Taufer, and B. Nicolae, "Towards Affordable Reproducibility Using Scalable Capture and Comparison of Intermediate Multi-Run Results," in *Middleware*, ser. Middleware '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 392–403. [Online]. Available: <https://doi.org/10.1145/3652892.3700780>
- [13] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Philadelphia, PA: SIAM, 2007.
- [14] J. E. Pearson, "Complex patterns in a simple system," *Science*, vol. 261, no. 5118, pp. 189–192, Jul. 1993.
- [15] L. Pottier, R. F. da Silva, H. Casanova, and E. Deelman, "Modeling the performance of scientific workflow executions on hpc platforms with burst buffers," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. New York: IEEE, 2020, pp. 92–103.
- [16] S. Zheng, M. Hoseinzadeh, and S. Swanson, "Zigurat: A tiered file system for Non-Volatile main memories and disks," in *17th USENIX Conference on File and Storage Technologies (FAST 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 207–219. [Online]. Available: <https://www.usenix.org/conference/fast19/presentation/zheng>
- [17] T. Wang, S. Oral, Y. Wang, B. Settlemeyer, S. Atchley, and W. Yu, "Burstmem: A high-performance burst buffer system for scientific applications," in *2014 IEEE International Conference on Big Data (Big Data)*. New York: IEEE, 2014, pp. 71–79.
- [18] U. Ayachit, A. C. Bauer, B. Boeckel, B. Geveci, K. Moreland, P. O'Leary, and T. Osika, "Catalyst revised: Rethinking the paraview in situ analysis and visualization API," in *High Performance Computing*. New York: Springer, Jun. 2021, pp. 484–494.
- [19] U. Ayachit, B. Whitlock, M. Wolf, B. Loring, B. Geveci, J. Lofstead, and S. Klasky, "SENSEI: A scalable and efficient end-to-end in situ framework," in *Proceedings of the Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization (ISAV)*. Salt Lake City, UT: IEEE, 2016, pp. 40–44.
- [20] A. Arroyo Ramo, J. A. Conejero, M. J. Perez-Quiles, and S. Hoyas, "Compact finite difference schemes: A review of fundamentals, applications, and practical implementation," *Mathematics*, vol. 14, no. 11, 2026. [Online]. Available: <https://www.mdpi.com/2227-7390/14/11/1958>
- [21] P. Bartholomew, G. Deskos, R. A. Frantz, F. N. Schuch, E. Lamballais, and S. Laizet, "Xcompact3d: An open-source framework for solving turbulence problems on a cartesian mesh," *SoftwareX*, vol. 12, p. 100550, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352711019303620>
- [22] WRF Users Page, "Wrf benchmark test - version 4.4," <https://www2.mmm.ucar.edu/wrf/users/benchmark/v44/>, 2022, accessed: 2025-04-14.
- [23] K. Ren, Q. Zheng, S. Patil, and G. Gibson, "Indexfs: scaling file system metadata performance with stateless caching and bulk insertion," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '14. IEEE Press, 2014, p. 237–248. [Online]. Available: <https://doi.org/10.1109/SC.2014.25>
- [24] R. Macedo, M. Miranda, Y. Tanimura, J. Haga, A. Ruhela, S. L. Harrell, R. T. Evans, J. Pereira, and J. Paulo, "Taming metadata-intensive hpc jobs through dynamic, application-agnostic qos control," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 47–61.
- [25] T. Fogal, F. Proch, A. Schiewe, O. Hasemann, A. Kempf, and J. Krüger, "Freeprocessing: Transparent in situ visualization via data interception," in *Eurographics Symposium on Parallel Graphics and Visualization (EGPGV)*. The Eurographics Association, 2014.
- [26] J. Cernuda, L. Logan, A. Gainaru, S. Klasky, J. Lofstead, A. Kougkas, and X.-H. Sun, "Hades: A context-aware active storage framework for accelerating large-scale data analysis," in *The 24th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, Philadelphia, May 6-9 2024, pp. 577–586.
- [27] F. Li and F. Song, "Instant: A runtime framework to orchestrate in-situ workflows," in *Euro-Par 2023: Parallel Processing*, J. Cano, M. D. Dikaiakos, G. A. Papadopoulos, M. Pericàs, and R. Sakellariou, Eds. Cham: Springer Nature Switzerland, 2023, pp. 199–213.
- [28] D. Marchant, M. Blomqvist, P. Jensen, I. Lilholm, and M. Nørgaard, "Delivering rules-based workflows for science," in *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2000–2008. [Online]. Available: <https://doi.org/10.1145/3624062.3624279>
- [29] M. Rynge, K. Vahi, E. Deelman, A. Mandal, I. Baldin, O. Bhide, R. Heiland, V. Welch, R. Hill, W. L. Poehlman, and F. A. Feltus, "Integrity protection for scientific workflow data: Motivation and initial experiences," in *Practice and Experience in Advanced Research Computing 2019: Rise of the Machines (Learning)*, ser. PEARC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3332186.3332222>